

DSE Control Plane — Metrics Reference

What every number on Admin Home and Analytics means, where it is computed, and whether it currently works.

Verified against the live control plane on 2026-09-01. Source of truth: `control-plane/api/src/analytics.ts` and the `console_rm` read model.

1. Where the numbers come from

Every figure on both screens is derived from three tables. Nothing on these screens is computed in the browser, and nothing is estimated unless this document says so explicitly.

```
DSE Postgres Console (SQLite) UI
audit_log ███
████ console_rm.runs_view ████████ runs ██████████
model_call_ledger ███ █████ analytics.ts ███ /api/metrics
work_items █████ console_rm.work_items_view ███ work_items █ /api/analytics
audit_log █████ console_rm.timeline_events ███ events ████████
```

A “run” is one agent turn, not one work item and not one HTTP call. Each row in `runs_view` carries the stage that produced it (planner, coder, tester, reviewer), its token counts and its cost in USD. The projector writes one row per metered model call.

Live shape of the data (2026-09-01)

Source	Content
runs_view	1,007 rows · \$860.28 · 12,048,429 tokens · 2026-07-24 → 2026-09-01
by stage	coder 502 (\$851.61) · tester 267 (\$4.51) · planner 204 (\$4.02) · reviewer 34 (\$0.14)
work_items_view	184 rows — blocked 100 · failed 77 · pr_ready 7
console_rm.baseline	0 rows — not configured
s	
status = done	never occurred (0 events)

Two of those lines govern most of what follows. **No work item has ever reached done**, and **no ROI baseline is configured**. Between them they explain every “—” on the dashboard.

2. Admin Home

2.1 Cost & Impact

Metric	How it is computed	Status
LLM spend last 30 days	Sum of <code>cost_usd</code> over every run started in the last 30 days. The sparkline buckets the same runs into 8 equal slices of that window. <code>analytics.ts:109</code>	WORKS
Delta “+1627% mo”	Percentage change against the <i>previous</i> 30 days. Tone is inverted — rising cost is red. <code>analytics.ts:37</code>	MISLEADING
Tokens last 7 days	Sum of <code>tokens_in</code> + <code>tokens_out</code> over runs started in the last 7 days, against the 7 days before it.	WORKS

Metric	How it is computed	Status
Agent cost	Sum of cost_usd over all runs ever — not the 30-day window used by the card beside it. analytics.ts:125	MISLEADING
Human estimate and savings	completed_tasks × baseline_hours × hourly_rate – agent_cost. Requires the baselines key; renders “—” without it, by design.	NO DATA

The 1627% delta is arithmetically correct and practically meaningless: the previous 30-day window contained almost no spend, so any activity produces a four-digit percentage. **Agent cost (\$860.28) and LLM spend (\$813.20) differ only because they cover different periods** — all-time versus 30 days — and the UI does not say so. Two numbers that look like they should match, and never will.

2.2 Development efficiency

Metric	How it is computed	Status
Headline (avg hours)	Mean of done_at – created_at across completed work items. done_at comes from the timeline event whose message starts with “→ done”.	NO DATA
Tasks completed	Count of work items with status done.	NO DATA
In flight	Count of status in {queued, running, pr_ready, review_feedback}. Currently 7 = the pr_ready items.	WORKS
Avg h / task	Same mean as the headline.	NO DATA
Failed	Count of work items with status failed. Currently 77.	WORKS
hours / task by category	Per repository: mean agent hours for completed items, against the configured human baseline.	NO DATA
“x% vs baseline”	$1 - \text{avg_agent_hours} / \text{baseline_hours}$. Needs both a completion and a baseline.	NO DATA

Five of the seven figures on this card cannot ever populate today. They are all gated on status = done, and an item only reaches done after a human merges its pull request. The DSE has produced 14 pull requests in its history and none were merged, so the card has never had an input. This is not a calculation bug — it is a definition question, addressed in §5.

2.3 Code quality / accuracy

Metric	How it is computed	Status
First-pass rate	(completed – reworked) / completed, where “reworked” means the item’s timeline contains a review-feedback event.	NO DATA
Merged	Count of status done — a proxy for merged, not a merge check . No GitHub merge state is read.	MISLEADING
Changes requested	Completed items whose timeline shows review rework.	NO DATA
Failed runs	Mislabelled. This is the count of failed <i>work items</i> (77), not failed runs. It is the same number shown as “Failed” on the efficiency card.	BROKEN
PRs open	Count of status in {pr_ready, review_feedback}. Currently 7.	WORKS
Donut legend (0% / 0% / 77%)	Broken. The legend appends “%” to a raw count. “Failed 77%” is 77 work items, not 77 per cent. admin-ui/src/app/page.tsx:316	BROKEN

2.4 Work item activity

Metric	How it is computed	Status
Active / Completed (7d) / Blocked / Failed	Direct counts over work_items by status.	WORKS
By status	Same counts, rendered as a list.	WORKS
By channel	Count of work items grouped by the intake channel that created them.	WORKS

These are the most trustworthy numbers on the page — plain counts over a table the projector maintains. Worth noting that **blocked (100) is the largest bucket on the whole dashboard and nothing anywhere explains why**: there is no breakdown of blocking reason on either screen.

3. Analytics

The three tabs read the same two tables as Admin Home, restricted to the selected date range and filters. The range applies to started_at for runs and to created_at/updated_at for work items, so a long-running item can appear in a window its runs do not.

3.1 Filters

Filter	Behaviour	Status
Date range	7d / 30d / 90d / QTD. QTD starts at the first day of the current calendar quarter.	WORKS
Repository	Filters work items by repo, then keeps only the runs belonging to those items.	WORKS
Agent	Filters by the runtime assigned to the item.	WORKS
Project / Task type	Inert. Both are hard-coded to a single option and filter nothing.	BROKEN

3.2 Cost tab

Metric	How it is computed	Status
Total cost	Sum of cost_usd over runs in range.	WORKS
Tokens	Sum of tokens_in + tokens_out over the same runs.	WORKS
Runs	Row count of those runs — i.e. agent turns, not work items.	WORKS
Cost / completed task	total_cost / completed_items. Renders “—” while nothing is completed, which is the current state.	NO DATA
Cost over time	The same runs bucketed into 8 equal slices of the range.	WORKS
Cost by repository	Each run is attributed to its work item's repository, then summed. Runs whose item is outside the filtered set fall into “unknown”.	WORKS
Cost by model	Wrong. Sums cost by the model name recorded on the run. Every stage is recorded as anthropic/claude-haiku, including the Coder, whose configured model is anthropic/claude. See §4.2.	BROKEN
Recent runs table	The last 6 runs in range, newest first.	WORKS

3.3 Efficiency tab

Metric	How it is computed	Status
Tasks completed	Count of status done in range.	NO DATA
Avg hours / task	Mean wall-clock from item creation to its “→ done” event.	NO DATA
Human baseline	Read straight from the configured baseline. “—” when unset.	NO DATA
Active runtimes	Distinct non-null runtimes assigned across items in range.	WORKS
Throughput	Completed items per bucket.	NO DATA
Avg hours by repository	Mislabelled. The title says “avg”, but the value is the <i>sum</i> of hours per repository — the aggregation used is a sum with no division.	BROKEN
Work items by status	Donut of done / in-progress / failed counts.	WORKS
Runtime table (Saved h)	$(\text{baseline_hours} - \text{avg_hours}) \times \text{completed}$, per runtime. An estimate, and empty without a baseline.	NO DATA

3.4 Quality tab

Metric	How it is computed	Status
First-pass rate	As on Admin Home: completed items with no review-rework event.	NO DATA
Changes requested	Completed items that were reworked.	NO DATA
Failed	Count of failed work items in range. Correctly labelled here, unlike Admin Home.	WORKS
PRs open	Count of {pr_ready, review_feedback}.	WORKS
Completions	Completed items per bucket.	NO DATA
Failures by repository	Failed item counts grouped by repository.	WORKS
Outcomes donut	Same count-rendered-as-percent defect as Admin Home.	BROKEN
Repository table — “Reworked” column	Wrong. Reads <code>status == review_feedback</code> (a current state) instead of the rework history, and it is gated behind a global counter, so it shows 0 whenever nothing was reworked anywhere.	BROKEN

4. Defects found, with the evidence

4.1 The donut legend renders counts as percentages

Where: `admin-ui/src/app/page.tsx:316` and `admin-ui/src/app/analytics/page.tsx:247` — both render `{q.value}%`, where `q.value` is a raw count.

Effect: the dashboard currently reads “Failed 77%”. The real figure is 77 work items out of 184, which is 42%. Every donut legend on both screens is affected.

Fix: divide by the segment total before rendering, or drop the “%” and label the column “items”. One line each.

Recommendation: fix.

4.2 Cost by model attributes the Coder's spend to the wrong model

Where: `model_call_ledger` records the model at metering time; the projector copies it into `runs_view.model` verbatim.

Evidence: the ledger labels every stage `anthropic/claude-haiku`, but the cost per run is not consistent with one model:

```
coder 467 runs $822.21 = $1.76 / run labelled haiku
planner 204 runs $4.02 = $0.02 / run labelled haiku
tester 267 runs $4.51 = $0.02 / run labelled haiku
```

A ninety-fold difference in cost per run between calls to the same model is not plausible. The Coder's configured model is `anthropic/claude`; the recorded name is a default that does not reflect the call. A further 35 Coder runs (\$29.40) carry no model at all.

Effect: “Cost by model” shows a single bar for a model that is not the one doing the spending. The chart is unusable for its purpose — choosing a cheaper model — and would actively mislead that decision.

Fix: record the model actually returned by the gateway response rather than the requested default.

Recommendation: **fix, or hide the chart until the label is trustworthy.** A wrong attribution is worse than an absent one.

4.3 “Failed runs” on Admin Home counts work items

Where: `analytics.ts:205` — the stat is built from the failed *work item* count while the label says runs. The same variable feeds “Failed” on the efficiency card, which is why both read 77.

Fix: either rename the label to “Failed items”, or compute it from `runs.status = 'error'`, which the table already carries. **Recommendation:** **rename.** The run-level figure is available but nobody has asked for it.

4.4 Two cost figures on one screen, over different windows

Card 01 shows a 30-day total; card 03 shows an all-time total. Both are labelled simply as cost.

Recommendation: **fix** — put “all time” on the second, or scope it to the same window.

4.5 “Avg hours by repository” sums instead of averaging

Where: `analytics.ts:321-323` aggregates with a sum and formats the result with an “h” suffix under a title that promises an average. With one completed item per repository the two agree, which is why it has never been noticed. **Recommendation:** **fix.**

4.6 Project and Task type filters do nothing

Both are populated with a single hard-coded option and are never applied to the query. **Recommendation:** **remove** until there is a project or task-type dimension in the data. A control that cannot change the result teaches the operator to distrust the controls that can.

5. The structural question: nothing is ever “done”

Eleven of the metrics above report “no data”, and they share one cause. Every efficiency and quality figure on both screens is gated on work items reaching status done, and done is only assigned after a human merges the pull request. In the platform's entire history no item has ever reached it.

The result is a dashboard whose two headline quality panels have never displayed a value, while the platform has in fact produced 14 pull requests. **The metrics are not wrong; the definition of “completed” is one the operator cannot satisfy by using the product.**

Three options, with what each costs

Option	What changes	Trade-off
Leave as is	Nothing. The panels populate the first time somebody merges a DSE pull request.	Honest, and useless until then. The dashboard cannot show progress on the thing being worked on.

Option	What changes	Trade-off
Add a second milestone	Track “delivered” (pull request opened and green) alongside “done” (merged). Efficiency measures creation-to-PR; quality keeps measuring merges.	Two definitions to explain, but each answers a real question: how fast the agent works, and how often its work is accepted.
Redefine done as PR-opened	One-line change to the status mapping.	Cheapest, and it destroys the quality panel: “merged without rework” stops meaning anything if nothing was merged. Not recommended.

Recommendation: the second option. It is the only one that lets the dashboard describe the platform's actual behaviour — which today is that it opens pull requests and does not get them merged — instead of showing a blank card that reads as “no activity”.

6. Summary and suggested order of work

#	Item	Action	Effort
1	Donut legend renders counts as percentages	Fix	1 line × 2 files
2	Cost by model attributed to the wrong model	Fix or hide	Metering change
3	“Failed runs” label counts work items	Rename	1 line
4	Agent cost is all-time beside a 30-day figure	Label or rescope	1 line
5	“Avg hours by repository” sums instead of averaging	Fix	1 line
6	Project / Task type filters do nothing	Remove	Delete two controls
7	Nothing ever reaches “done”	Product decision — \$5	Design first
8	“Blocked” is the biggest bucket and unexplained	Add a reason breakdown	New query

Items 1 and 3 through 6 are single-line corrections to display logic and carry no risk. Item 2 changes what is recorded and should be verified against a known model call before it is trusted. Item 7 is not an engineering task — it is a decision about what the product counts as finished, and every empty panel on the dashboard is waiting on it.